

HHI Research Journal

1 Project motivation and scope

The HHI project is built around the problem of converting non-physical human motion data into physically simulated humanoid motion. The long-term aim is to construct a dataset in which motions originally represented as kinematic human motion, we use AMASS as source, reproduced by a physically controlled humanoid in simulation.

The project is based mainly on three sources of prior work:

1. **AMP / ASE**: adversarial motion priors and reusable adversarial skill embeddings for physically simulated character control.
2. **PHC**: We adopt a similar target-motion imitation learning strategy.
3. **HUMOS**: used to generate motion variants ($2*64=128$) across body shapes and genders from AMASS dataset.

The key extension is **heteromorphic imitation**: the same motion content is associated with multiple humanoid morphologies, parameterized by gender and SMPL betas.

The morphology condition is represented as an 11-dimensional vector:

$$m = [g, \beta_1, \beta_2, \dots, \beta_{10}] \in \mathbb{R}^{11}, \quad (1)$$

where g is the gender code and $\beta_1, \dots, \beta_{10}$ are SMPL shape coefficients. The gender convention used in the project is:

$$g = \begin{cases} +1, & \text{male,} \\ -1, & \text{female,} \\ 0, & \text{not used, since HUMOS only support male/female..} \end{cases} \quad (2)$$

2 Relationship to Existing Projects

2.1 AMP/ASE

The source code of AMP/ASE provide the foundation for adversarial motion imitation. AMP introduces a discriminator that distinguishes policy-generated motion from reference motion. This allows the policy to receive style rewards from motion examples rather than relying only on hand-designed imitation terms.

In HHI, this adversarial imitation structure is extended so that the discriminator can also be conditioned on body morphology.

The standard discriminator can be written as:

$$D(x), \quad (3)$$

where x is an AMP observation. In HHI, the intended discriminator is:

$$D(x, m), \quad (4)$$

where m is the morphology vector. This means the discriminator should judge whether a motion is realistic for the given body shape, rather than judging motion independently of body shape.

2.2 PHC

PHC provides useful design patterns for humanoid control and progressive motion imitation. HHI initially borrowed several concepts from PHC, especially in observation design, transfer learning, and the possible use of a pretrained PHC-3 checkpoint. However, this transfer-learning direction was later abandoned, as the experiments did not show a meaningful improvement over training within the HHI setup.

3 Dataset and Morphology Generation

3.1 Body Shape Variations

The current data-generation plan uses 64 SMPL beta configurations and two genders. Therefore, each source motion can be expanded into:

$$64 \times 2 = 128 \quad (5)$$

morphology-specific variants.

For each source motion, the system should produce a set of target motions corresponding to different body shapes. This makes it possible to train a controller that is not bound to a single humanoid morphology.

3.2 HUMOS Output Structure

The HUMOS inference output is organized as one `.pt` file per input key. The expected file path is:

```
output/{keyid}.pt
```

Each file contains a dictionary named `motion_out`. The high-level structure is:

```
motion_out[gender_str][beta_key]
```

where `gender_str` belongs to:

```
{'male', 'female'}
```

Each `beta_key` entry contains SMPL-H time-series tensors:

Field	Shape	Meaning
<code>betas</code>	$[T, 10]$	SMPL shape parameters
<code>gender</code>	$[T, 1]$	Gender code
<code>root_orient</code>	$[T, 3]$	Root orientation
<code>pose_body</code>	$[T, 63]$	Body pose
<code>trans</code>	$[T, 3]$	Root translation
<code>offset_height</code>	scalar	Static grounding offset

The gender values are encoded as:

$$\text{female} = -1, \quad \text{male} = +1. \quad (6)$$

The `offset_height` value is computed from frame-0 vertices by taking the minimum value along the up-axis, with a safety margin. This helps ground the motion after retargeting.

3.3 Generated HUMOS Scale

The recorded HUMOS generation process used:

```
python humos/infer.py --cfg humos/configs/cfg_template_test.yml
```

The inference reads processed HumanML3D-style annotations:

```
humos/annotations/humanml3d/annotations_processed.json
```

The recorded total number of source motion sequences is:

$$22459. \quad (7)$$

Each source motion is expanded into 128 morphology variants, resulting from:

$$64 \text{ body shapes} \times 2 \text{ genders}. \quad (8)$$

The recorded generated output size is approximately:

$$778.054 \text{ GiB}. \quad (9)$$

3.4 Invalid Motion Filtering

We used ChatGPT and Grok to analyse the natural-language motion descriptions and identify motions that cannot be performed in an empty physical space. These include motions that require external support, furniture, objects, terrain structures, another person, animals, or other environmental interactions that are not represented in the current humanoid-only simulation environment.

A total of 1508 motions were filtered as invalid across 9 categories. The invalid motion list is stored in `data-processing/invalid_motions.txt`

The category-level counts are stored in: `data-processing/invalid_category_counts.txt`

The original motion set contains: 22459

motion sequences. After removing the 1508 invalid motions, the remaining valid set contains: $22459 - 1508 = 20951$

motions. These valid motions are stored in: `data-processing/valid_motions.txt`

The remaining 20951 motions will be used for the first main HHI training stage. This keeps the training environment simple: the humanoid is expected to move in an empty physical space without chairs, tables, walls, handheld objects, terrain constraints, or other interactive agents.

3.5 Difficulty-Based Curriculum Ordering

After invalid-motion filtering, the remaining valid motions are further ordered by estimated imitation difficulty. This step is used to construct a curriculum-learning schedule, so that the agent first learns relatively simple motions and later receives increasingly difficult motions.

We compute a motion difficulty score using:

```
scripts/compute_difficulty_score.py
```

The score is inspired by the Motion Difficulty Score (MDS) idea, where motions are ranked according to their intrinsic imitation difficulty. However, the current implementation uses a lightweight kinematic proxy rather than the full dynamics-based MDS formulation. For each motion, the script extracts root motion and joint-pose statistics, then computes:

$$d = 0.4v_{root}^{max} + 0.3r_{flight} + 0.2v_{dof}^{max} + 0.1\sigma^2(\|v_{root}\|), \quad (10)$$

where v_{root}^{max} is the maximum horizontal root velocity, r_{flight} is the fraction of frames with aerial upward motion, v_{dof}^{max} is the maximum joint velocity, and $\sigma^2(\|v_{root}\|)$ measures variation in root velocity magnitude.

The raw scores are min-max normalized across the motion set:

$$\hat{d}_i = \frac{d_i - d_{min}}{d_{max} - d_{min} + \epsilon}. \quad (11)$$

The valid motions are then sorted from easy to hard according to $\hat{d}_i$. The sorted list is divided into curriculum batches:

$$\mathcal{C} = \{C_1, C_2, \dots, C_N\}, \quad (12)$$

where each batch contains either 64 or 128 motions. The current script uses a batch size of 128, while 64 remains an alternative setting to be tested.

During training, the motion sampling pool can be expanded progressively:

$$\mathcal{M}_t = \bigcup_{k=1}^{K(t)} C_k, \quad (13)$$

where $K(t)$ increases over training time. This means early training uses only easier motions, while later stages include more difficult and dynamic motions. The curriculum strategy is borrowed from the difficulty-guided curriculum-learning idea in MDS-based humanoid imitation, where lower-difficulty motions are introduced first and harder motions are progressively added.

The computed difficulty scores are saved to:

`data/difficulty_scores.pkl`

and the sorted curriculum batches are saved to:

`data/curriculum_batches.json`

4 Humanoid XML Generation and Asset Loading

4.1 SMPLSim-Based Humanoid XML Generation

Before loading multiple humanoid assets into Isaac Gym, we first generate a set of morphology-specific MJCF XML files. This is done using a fork of SMPLSim, where SMPL body-shape parameters are converted into simplified humanoid XML assets compatible with the HHI training environment.

For each morphology, we sample a 10-dimensional SMPL beta vector:

$$\beta = [\beta_1, \beta_2, \dots, \beta_{10}] \in \mathbb{R}^{10}. \quad (14)$$

The beta parameters are sampled uniformly from the interval $[-3, 3]$:

$$\beta_i \sim \mathcal{U}(-3, 3), \quad i = 1, \dots, 10. \quad (15)$$

Uniform sampling is used to provide broad and even coverage of the SMPL shape space, rather than concentrating most samples near the neutral body shape. In the current setup, we generate 64 beta vectors. For each beta vector, two gender-specific humanoid assets are generated:

$$g \in \{\text{male}, \text{female}\}. \quad (16)$$

This produces:

$$64 \times 2 = 128 \quad (17)$$

humanoid XML files in total.

Each humanoid asset is generated through SMPLSim’s `SMPL_Robot` interface. The script uses the SMPL body model and exports the result as an MJCF XML file. To improve simulation stability, the generated humanoids use simplified collision and body settings: mesh collision is disabled, feet are replaced with simplified proxies, toes are removed, and hand joints are frozen. These choices reduce unstable contacts and unnecessary high-frequency joint behaviour during physics simulation.

The generated XML files follow the naming format:

`{gender}_{beta_key}_smpl.xml`

where `gender` is either `male` or `female`, and `beta key` is a deterministic hexadecimal identifier assigned to each sampled beta vector.

The generated XML files are saved under:

`ase/data/assets/mjcf/smpl/`

The sampled beta dictionary is also saved as:

`ase/data/assets/all_betas.pt`

This file provides the mapping between each `beta key` and its corresponding SMPL beta vector. Therefore, each humanoid XML file can be associated with its morphology-conditioning vector:

$$\mathbf{m} = [g, \beta_1, \dots, \beta_{10}] \in \mathbb{R}^{11}. \quad (18)$$

These generated XML assets form the morphology pool used by HHI. They are later loaded as multiple humanoid assets, allowing the policy, critic, and discriminator to be trained across different body shapes and genders.

4.2 Multiple Humanoid Assets

The HHI environment creates multiple humanoid assets corresponding to different gender and beta keys. Instead of loading only one humanoid XML, the environment loads a set of morphology-specific MJCF files.

The XML files follow a path pattern similar to:

`mjcf/smpl/{gender}_{beta_key}_smpl.xml`

The environment cycles these assets across simulation environments. Each environment is assigned a corresponding morphology key, and this mapping is stored for later motion sampling and shape-conditioned training.

4.3 XML Compiler Fix

A key stability fix was applied to the MJCF compiler settings:

```
compiler = self.tree.getroot().find("compiler")
compiler.attrib["coordinate"] = "local"
compiler.attrib["angle"] = "radian"
```

This fix is important because incorrect compiler settings can lead to unstable or incorrectly interpreted humanoid assets.

4.4 Base Humanoid Properties

The base humanoid uses the SMPL 24-body structure. The current recorded body and action structure is:

- Number of actuated DOFs: 69.
- Action dimension: 69.
- DOF observation size: 138.
- Self-observation size: 358.

The self-observation dimension is computed as:

$$1 + |B|(3 + 6 + 3 + 3) - 3 = 358, \quad (19)$$

where $|B| = 24$ is the number of body links. The final -3 removes the global root position components that should not be included directly.

The base humanoid also appends the 11-dimensional morphology vector to the policy observation.

5 Motion Library: MotionLibHUMOS

5.1 Purpose

MotionLibHUMOS is responsible for loading HUMOS-generated target motions and matching them with the correct humanoid morphology. Unlike a single-shape motion library, this motion library must handle multiple gender and beta combinations.

5.2 Skeleton Tree Construction

For every loaded (`gender`, `beta_key`) pair, the motion library constructs a corresponding skeleton tree from the morphology-specific XML file. This ensures that motion loading, forward kinematics, and imitation targets are aligned with the correct body shape.

5.3 Motion Loading

During loading, each motion stores or derives:

- gender;
- beta vector;
- beta key;

- root translation;
- root orientation;
- body pose;
- DOF positions;
- DOF velocities;
- rigid body positions and rotations;
- key body positions;
- the 11-dimensional morphology vector.

The motion library also applies a height correction step through `fix_trans_height`. This uses the SMPL parser and the current beta/gender configuration to estimate the lowest vertex height and correct the translation accordingly.

5.4 Shape-Matched Motion Sampling

A key design requirement is that each simulated humanoid should sample target motions matching its own morphology. Therefore, the motion library maintains a mapping:

```
beta_key_motion_id_mapping[(gender, beta_key)] -> list[motion_id]
```

When the environment resets, it requests motions using the environment’s assigned morphology key. This prevents the male humanoid with one beta shape from imitating a target generated for a different body shape.

The sampling operation can be summarized as:

$$\text{motion_id} \sim p(\text{motion} \mid g, \beta_key). \quad (20)$$

This is one of the central corrections needed for shape-conditioned imitation learning.

6 Current HHI Environment Design

6.1 Environment Class

The current environment class is:

```
HumanoidHHI(Humanoid)
```

It extends the base humanoid task with target-motion imitation, AMP observations, HUMOS motion loading, morphology-aware sampling, target markers for visualization, power penalty, and optional smoothness penalty.

6.2 Reset Procedure

During reset, the environment:

1. resets progress, reset, and termination buffers;
2. obtains the environment’s assigned (`gender`, `beta_key`);
3. samples a target motion matching this morphology;
4. samples a random motion time during training;

5. uses time zero during target-marker playback mode;
6. extracts the reference motion state;
7. resets the humanoid root, body, and DOF state to match the reference frame;
8. recomputes policy observations;
9. resets previous-action buffers if smoothness reward or action filtering is enabled.

This reset logic is important because the humanoid must start from a physically and morphologically consistent state.

6.3 Post-Physics Step

After every physics step, the environment:

1. increments progress buffers;
2. refreshes simulator tensors;
3. computes target-task observations;
4. obtains reference motion states;
5. computes imitation reward;
6. computes reset conditions;
7. updates AMP observation history;
8. flattens AMP observations;
9. exports AMP observations and AMP shape vectors through `extras`.

The AMP outputs are:

```
self.extras["amp_obs"] = amp_obs_flat
self.extras["amp_shape"] = self._betas_env
```

This makes the fake policy samples available to the agent and discriminator.

7 Observation Design

7.1 Current Main-Branch Policy Observation

The current main implementation uses a policy observation of 585 dimensions:

$$358 + 216 + 11 = 585. \tag{21}$$

The components are:

Component	Dimension
Self observation	358
Task observation v7	216
Morphology vector	11
Total	585

The task observation v7 uses 24 key bodies, with 9 dimensions per key body:

$$24 \times 9 = 216. \quad (22)$$

The final policy observation layout is:

$$o = [o_{self}, o_{task}, m]. \quad (23)$$

In the current network builder, this is treated as:

```
obs[:, :574] -> state and task observation
obs[:, 574:] -> morphology condition
```

where:

$$574 + 11 = 585. \quad (24)$$

7.2 Potential Beta Normalization Issue

The current environment computes a normalized beta tensor:

```
betas_norm = betas.clone()
betas_norm[:, 1:] /= 3.0
```

However, the current observation concatenation appends the raw **betas**, not **betas_norm**:

```
obs = torch.cat([obs, task_obs], dim=-1)
obs = torch.cat([obs, betas], dim=-1)
```

This should be treated as an open implementation detail. If the FiLM conditioners expect stable scale, normalized beta values may be preferable. If checkpoint compatibility expects raw beta values, raw betas may be acceptable. The important point is that this choice should be explicit rather than accidental.

7.3 PHC-Transfer Policy Observation

A later transfer-learning design uses a different observation layout. The PHC-compatible source observation is recorded as 934 dimensions. Adding the 11-dimensional morphology vector gives:

$$934 + 11 = 945. \quad (25)$$

This 945-dimensional observation is associated with PHC transfer learning and should not be confused with the current 585-dimensional main-branch implementation.

8 AMP Observation Design

8.1 Current Main-Branch AMP Observation

The current main-branch AMP observation uses 292 dimensions per step and 10 history steps:

$$292 \times 10 = 2920. \quad (26)$$

The per-step AMP observation can be decomposed as:

$$13 + 138 + 69 + 72 = 292. \quad (27)$$

The components are:

Component	Dimension
Root features	13
DOF observation	138
DOF velocity	69
Key body position features	72
Per-step total	292
History steps	10
Flattened total	2920

The current discriminator does not append shape directly to the AMP observation. Instead, the discriminator receives:

```
amp_obs
amp_shape
```

as separate inputs.

8.2 PHC-Compatible AMP Observation

The PHC-transfer notes describe a PHC-compatible AMP observation of 196 dimensions per step and 10 history steps:

$$196 \times 10 = 1960. \quad (28)$$

This was introduced to match the pretrained PHC discriminator input size. It involves using a smaller discriminator-relevant observation set rather than the full current 292-dimensional per-step AMP observation.

Therefore, the project currently has two relevant AMP observation regimes:

Stage	Per-Step AMP Dim	Flattened AMP Dim
Current main HHI	292	2920
PHC-transfer design	196	1960

These should be kept separate in the journal unless the implementation is later unified.

9 Reward Design

9.1 Imitation Reward

The environment uses a target-motion imitation reward with position, rotation, velocity, and angular velocity terms. The reward can be written as:

$$r_{imit} = w_p e^{-k_p E_p} + w_r e^{-k_r E_r} + w_v e^{-k_v E_v} + w_\omega e^{-k_\omega E_\omega}. \quad (29)$$

The recorded reward settings are:

```
{
  "k_pos": 50,
  "k_rot": 30,
  "k_vel": 0.2,
  "k_ang_vel": 0.2,
  "w_pos": 0.45,
  "w_rot": 0.25,
  "w_vel": 0.15,
```

```

    "w_ang_vel": 0.15
}

```

9.2 Power Penalty

A power penalty is enabled to discourage excessive actuation:

$$r_{power} = -\lambda_{power} \sum_j |\tau_j \dot{q}_j|. \quad (30)$$

The recorded default coefficient is:

$$\lambda_{power} = 0.00005. \quad (31)$$

The penalty is ignored during the first few frames after reset to avoid punishing initial stabilization:

```

power_reward[progress_buf <= 3] = 0

```

9.3 Smoothness Reward

A smoothness penalty can be used to reduce jitter:

$$r_{smooth} = -\lambda_{smooth} \|a_t - a_{t-1}\|^2. \quad (32)$$

The recorded default coefficient is:

$$\lambda_{smooth} = 0.005. \quad (33)$$

The current project notes indicate that smoothness reward is enabled, while PD tuning and action filtering are currently disabled by default.

10 Action Filtering and PD Tuning

The base humanoid contains support for optional action filtering and PD gain tuning.

10.1 Action Filtering

The action filter uses an exponential moving average:

$$a_t^f = \alpha a_{t-1}^f + (1 - \alpha) a_t. \quad (34)$$

The suggested range from the notes is:

$$\alpha \in [0.8, 0.9]. \quad (35)$$

This is intended to reduce high-frequency action changes and therefore reduce visible jitter.

10.2 PD Gain Tuning

The notes record a possible jitter-mitigation strategy:

- increase damping K_d by approximately 1.5 to 2.0 times;
- reduce stiffness K_p by approximately 0.7 to 0.9 times;
- add extra damping for arms if arm jitter remains visible.

This has not yet been fully activated in the current training setup.

11 Shape-Conditioned Network Design

11.1 Overall Design

The HHI network uses morphology-conditioned FiLM modulation for the actor, critic, and discriminator. The morphology vector is not simply appended to every layer. Instead, it is passed through conditioning networks that produce FiLM parameters.

For a hidden activation h_i , FiLM modulation can be written as:

$$h'_i = h_i \odot \gamma_i(m) + \beta_i(m), \quad (36)$$

where $\gamma_i(m)$ and $\beta_i(m)$ are generated from the morphology vector.

11.2 Actor

The actor receives the 585-dimensional policy observation but internally splits it into:

<code>state/task observation: 574 dims</code> <code>morphology condition: 11 dims</code>

The actor trunk processes only the 574-dimensional state/task observation. The 11-dimensional morphology vector is processed by a conditioning MLP that generates FiLM parameters for actor hidden layers.

The actor input split is:

$$o = [s, m], \quad s \in \mathbb{R}^{574}, \quad m \in \mathbb{R}^{11}. \quad (37)$$

The action output dimension is:

$$\dim(a) = 69. \quad (38)$$

11.3 Critic

Although an earlier comment suggested that the critic was unchanged, the current implementation also reconstructs the critic trunk and applies morphology-conditioned FiLM modulation.

The critic uses the same observation split:

$$V(s, m). \quad (39)$$

This is reasonable because body morphology can affect physical difficulty, balance, and achievable imitation quality.

11.4 Discriminator

The discriminator receives AMP motion features and the morphology vector separately:

<code>eval_disc(amp_obs, amp_shape)</code>
--

The AMP observation is passed through the discriminator MLP, while `amp_shape` is passed through a discriminator conditioning network that generates FiLM parameters.

The shape-conditioned discriminator is therefore:

$$D(x, m), \quad (40)$$

where x is the AMP observation and m is the morphology condition.

11.5 Discriminator Objective

The discriminator receives three categories of samples:

- fake policy samples: `amp_obs`, `amp_shape`;
- replay fake samples: `amp_obs_replay`, `amp_shape_replay`;
- real demonstration samples: `amp_obs_demo`, `amp_shape_demo`.

The intended discriminator loss can be summarized as:

$$\mathcal{L}_D = \frac{1}{2}\text{BCE}(D(x_{fake}, m_{fake}), 0) + \frac{1}{2}\text{BCE}(D(x_{real}, m_{real}), 1) + \text{regularizers}. \quad (41)$$

The key conceptual requirement is that real and fake samples must both be shape-conditioned:

$$x_{real} \sim p_{data}(x \mid m), \quad x_{fake} \sim p_{\pi}(x \mid m). \quad (42)$$

12 HHI Agent Training Loop

12.1 High-Level Loop

The HHI agent follows an AMP/PPO-style training loop:

1. Roll out the current policy for a fixed horizon.
2. Store PPO data, AMP observations, and AMP shape vectors.
3. Sample real demonstration AMP observations from the environment.
4. Sample replay AMP observations from the fake replay buffer.
5. Train actor, critic, and discriminator.
6. Store new fake AMP samples into the replay buffer.

12.2 AMP Shape Propagation

The important HHI-specific extension is that every AMP observation must carry its corresponding morphology vector. During rollout, the agent stores:

<code>amp_obs</code> <code>amp_shape</code>
--

During dataset preparation, the agent also prepares:

<code>amp_obs_demo</code> <code>amp_shape_demo</code> <code>amp_obs_replay</code> <code>amp_shape_replay</code>
--

This ensures that the discriminator can compare fake and real samples under the correct morphology condition.

12.3 Replay Buffer

The fake replay buffer stores both AMP observations and shape vectors. This avoids the incorrect situation where a replayed motion feature is evaluated under the wrong body morphology.

13 PHC Transfer Learning Stage

13.1 Motivation

Training from scratch on a single motion can produce motion that roughly follows the target but still shows jitter, especially in the arms. Transfer learning from a PHC checkpoint is intended to provide a stronger initial policy and discriminator, allowing the HHI model to learn shape-conditioned imitation more efficiently.

13.2 Checkpoint Contents

The PHC checkpoint contains actor, critic, discriminator, normalization statistics, optimizer state, and training metadata. Important entries include:

```
running_mean_std
reward_mean_std
amp_input_mean_std
model
epoch
optimizer
frame
last_mean_rewards
env_state
```

The actor action dimension is inferred from the sigma vector:

$$\dim(a) = 69. \quad (43)$$

13.3 PHC Network Size

The PHC actor/critic MLP structure discussed in the notes uses:

$$[2048, 1536, 1024, 1024, 512, 512]. \quad (44)$$

The discriminator structure discussed in the notes uses:

$$[1024, 512]. \quad (45)$$

The actor/critic activation is SiLU, while the discriminator uses ReLU.

13.4 Selective Weight Loading

The transfer-learning strategy is to load only compatible parameters from the PHC checkpoint. Parameters are copied when both name and shape match. New HHI-specific modules, such as FiLM conditioning layers, remain randomly initialized.

This avoids forcing incompatible checkpoint tensors into a modified architecture.

The intended logic is:

```
for key, src_tensor in ckpt_model.items():
    if key in current_model and current_model[key].shape == src_tensor.shape:
        current_model[key].copy_(src_tensor)
    else:
        skip(key)
```

13.5 Normalization Statistics

Normalization statistics must also be loaded selectively. The notes record that:

- `amp_input_mean_std` can be loaded when AMP observation dimensions match;
- `running_mean_std` must be skipped if policy observation dimensions differ;
- `reward_mean_std` is intentionally skipped.

A recorded mismatch occurred when trying to load 934-dimensional running statistics into a 945-dimensional observation space. This is expected if morphology is appended to the PHC observation.

13.6 Ideal Call Site

The proposed call site for loading PHC weights is inside:

```
PHCAgent._init_train()
```

The intended order is:

```
super()._init_train()  
_load_pretrained_phc3()  
_init_amp_demo_buf()
```

This ensures that the model and normalization objects already exist, while training has not yet begun.

14 Curriculum and Batch Design

14.1 Motion-ID Batch Size

The notes distinguish between source motion IDs and morphology-expanded variants. Since each motion ID expands to 128 morphology variants, loading too many source motion IDs can quickly create a very large effective motion library.

The suggested initial batch size is:

$$128 \text{ motion IDs,} \tag{46}$$

or:

$$64 \text{ motion IDs} \tag{47}$$

if memory is insufficient.

14.2 Motion Difficulty Score

A proposed curriculum uses motion difficulty scoring. The recorded features are:

- maximum horizontal root velocity;
- flight ratio;
- maximum DOF velocity;
- kinetic variance.

The proposed score is:

$$\begin{aligned} \text{difficulty} = & 0.4 \text{norm}(\text{max_root_hvel}) \\ & + 0.3 \text{norm}(\text{flight_ratio}) \\ & + 0.2 \text{norm}(\text{max_dof_vel}) \\ & + 0.1 \text{norm}(\text{kinetic_var}). \end{aligned} \tag{48}$$

The recorded script name is:

`scripts/compute_difficulty_score.py`

This curriculum is intended to avoid training immediately on the full, highly diverse, and noisy motion set.

15 Jitter Problem and Mitigation

15.1 Observed Problem

Early training on a single target motion produced a humanoid that roughly followed the motion but had visible jitter, especially in the arms. Similar jitter appeared both in scratch training and transfer-learning experiments.

This suggests that the issue is not only network initialization. It may also involve:

- overfitting to a single clip;
- insufficient motion diversity;
- action high-frequency noise;
- PD gain settings;
- weak smoothness regularization;
- mismatch between target kinematics and physically feasible dynamics.

15.2 Metrics for Jitter

Possible metrics include:

- mean joint jerk;
- power spectral density of joint velocities or accelerations;
- torque variance;
- action difference magnitude;
- fall rate;
- per-motion success rate.

Mean joint jerk can be approximated by finite differences:

$$\text{jerk} = \frac{1}{T} \sum_t \|\ddot{\mathbf{q}}_t\|. \tag{49}$$

15.3 Mitigation Options

The current and proposed mitigation options are:

1. enable or tune smoothness reward;
2. apply action EMA filtering;
3. tune PD stiffness and damping;
4. train on multiple target motions instead of a single clip;
5. use curriculum learning;
6. use PHC transfer learning;
7. quantify jitter before and after each change.

16 Current Implementation Status

The current state can be summarized as follows:

- HUMOS output generation has been completed at large scale.
- Each source motion can be expanded to 128 morphology variants.
- Humanoid XMLs are generated and loaded per gender/beta key.
- The environment supports morphology-specific assets.
- The motion library supports morphology-specific motion sampling.
- The current policy observation is 585-dimensional on the main implementation path.
- The current AMP observation is 2920-dimensional on the main implementation path.
- The actor, critic, and discriminator are shape-conditioned through FiLM.
- The agent propagates AMP shape vectors for fake, replay, and real samples.
- A PHC-transfer path has been designed with 945-dimensional policy observation and 1960-dimensional AMP observation.
- Smoothness reward support exists and is currently enabled.
- PD tuning and action filtering exist but are currently disabled by default.

17 Open Issues

17.1 Observation-Dimension Split

There are two observation regimes in the notes:

Stage	Policy Obs	AMP Obs	Purpose
Current main HHI	585	2920	Current shape-conditioned training path
PHC-transfer path	945	1960	PHC checkpoint compatibility

These should remain clearly separated unless a later implementation unifies them.

17.2 Beta Normalization

The current code computes `betas_norm` but appends raw `betas`. This needs to be clarified before long training runs. The choice affects the scale seen by FiLM conditioning networks.

17.3 Sitting and Environment-Supported Motions

Motions requiring chairs, tables, walls, objects, terrain, or other people are difficult to imitate in a humanoid-only environment. These should either be filtered or assigned to future environment-aware tasks.

17.4 Jitter Evaluation

Qualitative video inspection is useful but insufficient. The project should add quantitative jitter diagnostics, including jerk, torque variance, and high-frequency spectral energy.

18 Suggested Run Log Template

Each important run should be recorded using the following format:

```
Date:
Commit hash:
Config files:
Checkpoint initialized from:
Observation dim:
AMP observation dim:
Number of envs:
Number of motion IDs:
Number of morphology variants:
Reward weights:
Power coefficient:
Smoothness settings:
PD settings:
Action filtering settings:
Training duration:
Mean return:
Task reward:
Discriminator reward:
Discriminator real/fake accuracy:
Fall rate:
Per-motion success rate:
Qualitative result:
Failure modes:
Next action:
```

19 Next Technical Steps

The immediate next steps are:

1. decide whether policy morphology input should use raw or normalized beta values;
2. verify which branch is used for the next training run: current HHI path or PHC-transfer path;
3. add run-level logging for observation dimensions, AMP dimensions, and morphology settings;

4. begin training with a controlled subset of motion IDs;
5. record jitter metrics in addition to video output;
6. keep sitting and environment-supported motions out of the first stable training curriculum.